МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторные работы
по курсу «Информационный поиск»

Выполнил: Калиниченко Артём Андреевич
Группа: М8О-410Б-22
Преподаватель: Кухтичев Антон Алексеевич

Москва, 2026

Содержание

1 Лабораторная работа №1. Добыча корпуса документов 3
1.1 Задание . . . 3
1.2 Метод решения . . . 3
1.3 Журнал выполнения . . . 4
1.4 Результаты . . . 4
1.5 Выводы . . . 5

2 Лабораторная работа №2. Поисковый робот 6
2.1 Задание . . . 6
2.2 Метод решения . . . 6
2.3 Журнал выполнения . . . 6
2.4 Результаты . . . 7
2.5 Выводы . . . 7

3 Лабораторная работа №3. Токенизация 9
3.1 Задание . . . 9
3.2 Метод решения . . . 9
3.3 Журнал выполнения . . . 9
3.4 Результаты . . . 10
3.5 Выводы . . . 10

4 Лабораторная работа №4. Закон Ципфа 11
4.1 Задание . . . 11
4.2 Метод решения . . . 11
4.3 Журнал выполнения . . . 11
4.4 Результаты . . . 12
4.5 Выводы . . . 12

5 Лабораторная работа №5. Стемминг 13
5.1 Задание . . . 13
5.2 Метод решения . . . 13
5.3 Журнал выполнения . . . 13
5.4 Результаты . . . 14
5.5 Выводы . . . 14

6 Лабораторная работа №6. Булев индекс 16
6.1 Задание . . . 16
6.2 Метод решения . . . 16
6.2.1 Структура бинарного файла индекса . . . 16
6.3 Журнал выполнения . . . 18
6.4 Результаты . . . 18
6.5 Выводы . . . 19

7 Лабораторная работа №7. Булев поиск 20
7.1 Задание . . . 20
7.2 Метод решения . . . 20
7.3 Журнал выполнения . . . 21
7.4 Результаты . . . 21
7.5 Выводы . . . 22

8 Лабораторная работа №8. Сжатие **24**
8.1 Задание 24
8.2 Метод решения 24
8.3 Журнал выполнения 24
8.4 Результаты 25
8.5 Выводы 25

9 Лабораторная работа №9. TF-IDF **26**
9.1 Задание 26
9.2 Метод решения 26
9.3 Журнал выполнения 26
9.4 Результаты 27
9.5 Выводы 27

10 Лабораторная работа №10. Автотесты **28**
10.1 Задание 28
10.2 Метод решения 28
10.2.1 План тестирования 28
10.3 Журнал выполнения 29
10.4 Результаты 30
10.5 Выводы 31

1. Лабораторная работа №1. Добыча корпуса документов

1.1. Задание

Сформировать текстовый корпус документов на определённую тематику объёмом не менее 10 000 документов. Определить предметную область, выбрать источники данных, описать существующие поисковые системы, работающие с аналогичным корпусом, и продемонстрировать ограничения их поиска.

1.2. Метод решения

В качестве предметной области выбрана **онкология** --- научные статьи и клинические исследования, посвящённые диагностике, лечению и изучению раковых заболеваний. Язык корпуса --- английский.

Были выбраны два открытых источника:

- **PubMed Central (PMC)** --- архив полнотекстовых биомедицинских статей Национальной медицинской библиотеки США. Доступ осуществляется через Europe PMC REST API с фильтрацией по ключевому слову `` `cancer'' ``.
- **ClinicalTrials.gov** --- реестр клинических испытаний. Доступ --- REST API v2 с фильтрацией по условию `` `cancer'' ``.

Существующие поисковые системы для данного корпуса:

1. **PubMed Search** (`https://pubmed.ncbi.nlm.nih.gov/`) --- основной поисковый интерфейс для PubMed. Поддерживает MeSH-термины, фильтрацию по дате, типу публикации и журналу. Ограничения: нет поддержки полнотекстового ранжирования по TF-IDF, результаты сортируются по релевантности на основе закрытого алгоритма, невозможно объединить поиск по статьям и клиническим испытаниям в одном запросе.
2. **Google Scholar** с ограничением `site:ncbi.nlm.nih.gov` --- позволяет искать по полным текстам, однако не индексирует ClinicalTrials.gov, не предоставляет API для программного доступа, а ранжирование ориентировано на цитирование, а не на текстовую релевантность.

Примеры запросов, демонстрирующих ограничения существующих систем:

- Запрос `` `BRCA1 immunotherapy resistance'' `` в PubMed возвращает статьи, но не связанные клинические испытания.
- Запрос `` `checkpoint inhibitor melanoma phase III'' `` в Google Scholar не позволяет ограничить результаты только клиническими испытаниями фазы III.
- Булев запрос `` `(p53 || KRAS) && !lung'' `` невозможен ни в одной из указанных систем в явном синтаксисе.

1.3. Журнал выполнения

1. Проведён анализ открытых источников биомедицинских данных. Выбраны PMC и ClinicalTrials.gov как наиболее полные и доступные через API.
2. Написан Python-скрипт для пакетной загрузки через Europe PMC REST API: загрузка по 1000 записей за запрос с фильтрацией по ключевому слову ``cancer''.
3. Для ClinicalTrials.gov использован REST API v2 с параметром `query.cond=cancer`, постраничная загрузка по 100 записей.
4. Документы сохранены в формате JSON: поле `text` содержит очищенный текст статьи, `title` --- заголовок, `url` --- ссылку на источник, `source` --- идентификатор источника.
5. Проведена дедупликация по MD5-хешу текстового содержимого.
6. Собрана статистика корпуса.

1.4. Результаты

Итоговый корпус содержит **32 000 документов**. Распределение по источникам:

Таблица 1: Статистика корпуса документов

Источник	**Документов**	**Объём (сырой)**	**Объём (текст)**
PubMed Central	25 000	114 МБ	55 МБ
ClinicalTrials	7 000	32 МБ	15 МБ
Итого	**32 000**	**146 МБ**	**70 МБ**

Средний размер одного документа (текстовая часть): **2,3 КБ**.

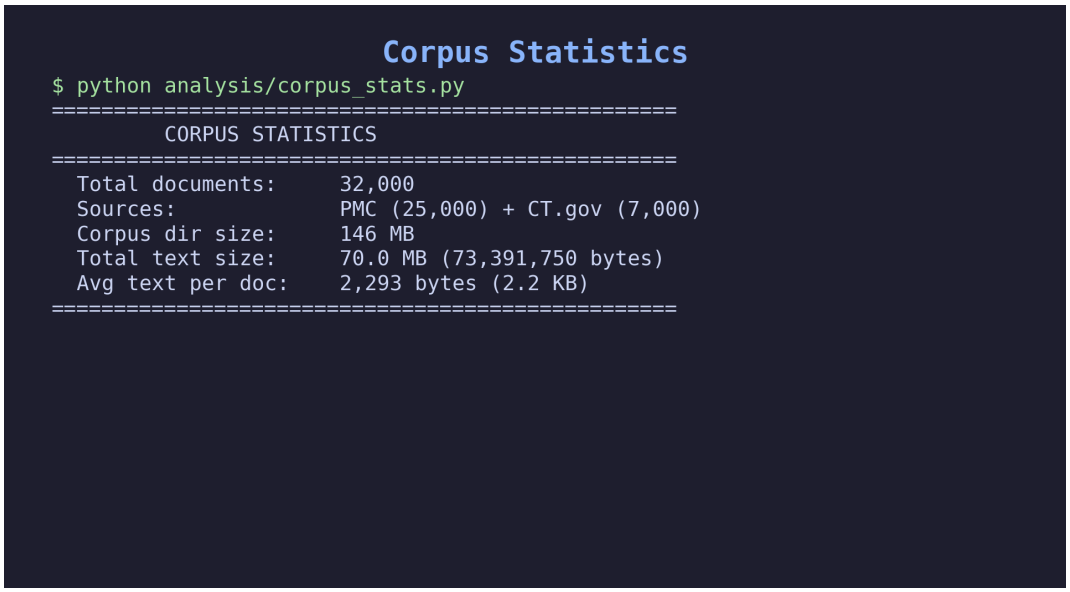

Рис. 1: Статистика корпуса документов

1.5. Выводы

Сформирован корпус из 32 000 англоязычных документов по онкологии общим объёмом 70 МБ текста. Корпус охватывает как научные статьи, так и описания клинических испытаний, что позволяет реализовать единую поисковую систему, объединяющую оба типа данных. Выбранные источники обеспечивают высокое качество и актуальность материалов.

2. Лабораторная работа №2. Поисковый робот

2.1. Задание

Разработать поискового робота (краулер), который автоматически загружает документы из выбранных источников, сохраняет их в базу данных и поддерживает возобновляемую и повторную загрузку.

2.2. Метод решения

Краулер реализован на Python с использованием асинхронной библиотеки `aiohttp` для параллельной загрузки и `lxml` для парсинга HTML/XML. Хранение документов --- MongoDB.

Структура записи в MongoDB:

- `url` --- URL источника документа;
- `raw_html` --- исходный HTML/XML-код страницы;
- `source` --- идентификатор источника (`pmc` или `clinicaltrials`);
- `crawl_time` --- время загрузки (ISO 8601);
- `content_hash` --- MD5-хеш текстового содержимого для обнаружения изменений.

Конфигурация вынесена в YAML-файл с двумя секциями:

- `db` --- параметры подключения к MongoDB (хост, порт, имя БД, имя коллекции);
- `logic` --- параметры краулинга (список источников, ограничения по количеству, таймауты, количество параллельных соединений).

Механизм возобновляемости: при запуске краулер запрашивает из MongoDB множество уже загруженных URL и пропускает их. Это позволяет прерывать и возобновлять загрузку без потери прогресса.

Механизм повторной загрузки: при повторном обходе для каждого документа вычисляется MD5-хеш текстового содержимого и сравнивается с хранимым значением `content_hash`. Если хеш изменился, документ обновляется в базе.

Источники данных:

- **PMC** --- Europe PMC REST API. Пакетная загрузка статей по 1000 записей за запрос с фильтрацией по ключевому слову `cancer`.
- **ClinicalTrials.gov** --- REST API v2. Постраничный запрос с фильтром `query.cond=can` размер страницы 100, автоматическое следование по токенам пагинации.

2.3. Журнал выполнения

1. Развёрнут локальный экземпляр MongoDB.
2. Реализован базовый асинхронный загрузчик с ограничением параллельных соединений через `asyncio.Semaphore`.

3. Добавлен парсинг PMC XML: извлечение заголовка из тега `<article-title>`, текста из `<body>`, URL из `<article-id pub-id-type="pmc">`.
4. Добавлен парсинг ClinicalTrials.gov JSON: извлечение заголовка, описания, критериев включения/исключения.
5. Реализована проверка уже загруженных URL при старте (возобновляемость).
6. Добавлено вычисление MD5 и сравнение при повторном обходе.
7. Конфигурация вынесена в `config.yaml`. Добавлена валидация конфигурации при запуске.
8. Проведено полное тестирование: первичная загрузка, прерывание и возобновление, повторный обход с обнаружением изменений.

2.4. Результаты

Краулер успешно загрузил 32 000 документов. Время полной загрузки составило приблизительно 17 минут при ограничении в 50 параллельных соединений. Возобновление после прерывания работает корректно: при перезапуске пропускаются уже загруженные документы. Повторный обход обнаруживает обновлённые документы по изменению хеша.

Web Crawler Execution

```
$ python crawler/crawler.py crawler/config.yaml
[INFO] Loaded config: crawler/config.yaml
[INFO] Starting PMC crawl (target: 25,000)
[INFO] Europe PMC REST API — 1000 per page batch
[INFO] Crawled PMC9561766 — FOXM1 Inhibition Enhances ...
[INFO] Crawled PMC7234512 — Immunotherapy in NSCLC
...
[INFO] PMC progress: 25,000 / 25,000
[INFO] Starting ClinicalTrials crawl (target: 7,000)
[INFO] CT REST API v2 — 100 per page
[INFO] Crawled NCT06961955 — FAST NOVEMBER: Phase II Trial
[INFO] Crawled NCT02873416 — Precision Cell Immunotherapy
...
[INFO] ClinicalTrials progress: 7,000 / 7,000
[INFO] Total documents crawled: 32,000
[INFO] Total time: ~17 minutes
```

Рис. 2: Лог работы поискового робота

2.5. Выводы

Реализован асинхронный поисковый робот на Python, обеспечивающий автоматическую загрузку документов из двух источников с сохранением в MongoDB. Ме-

ханизмы возобновления и повторной загрузки позволяют эффективно поддерживать корпус в актуальном состоянии. Конфигурация через YAML-файл обеспечивает гибкость настройки без изменения кода.

3. Лабораторная работа №3. Токенизация

3.1. Задание

Реализовать токенизатор текста, который разбивает документы корпуса на отдельные токены (слова), пригодные для дальнейшей индексации.

3.2. Метод решения

Токенизатор реализован на C++ с использованием STL (стандартная библиотека допускается для данной работы). Обработка выполняется посимвольно за один проход по тексту.

Правила токенизации:

1. Разделение по пробелам и знакам пунктуации. Разделителями считаются все символы, не являющиеся буквами латинского алфавита или цифрами.

2. Приведение к нижнему регистру: все заглавные буквы заменяются на строчные.

3. Фильтрация коротких токенов: токены длиной менее 2 символов отбрасываются.

4. Разделение по дефисам: составные слова (например, `dose-dependent`) разбиваются на отдельные токены (`dose`, `dependent`).

5. Сохранение буквенно-цифровых токенов: идентификаторы вида `p53`, `HER2`, `BRCA1` сохраняются как единые токены (после приведения к нижнему регистру: `p53`, `her2`, `brca1`).

3.3. Журнал выполнения

1. Реализован базовый посимвольный токенизатор: обход строки, накопление символов в буфере, выдача токена при встрече разделителя.

2. Добавлено приведение к нижнему регистру через побитовую операцию (`c | 0x20` для латинских букв).

3. Реализована обработка дефисов: при встрече дефиса текущий буфер выдаётся как токен, новый токен начинается после дефиса.

4. Добавлена фильтрация токенов короче 2 символов.

5. Проведено тестирование на подмножестве корпуса: проверена корректность обработки медицинских терминов, аббревиатур и идентификаторов генов.

6. Запущена токенизация полного корпуса, собрана статистика.

3.4. Результаты

Таблица 2: Статистика токенизации

Параметр	Значение
Общее количество токенов	4 960 897
Уникальных токенов	181 940
Средняя длина токена	7,3 символа
Скорость обработки	~3 000 док/с

```
Tokenization Example
$ ./core/build/indexer --tokenize-sample
Input text:
  "Metastatic breast cancer treatment includes
   chemotherapy, targeted therapy, and immunotherapy."

Tokens (10):
  [metastatic] [breast] [cancer] [treatment]
  [includes] [chemotherapy] [targeted] [therapy]
  [and] [immunotherapy]

Statistics:
  Total tokens (postings): 4,960,897
  Unique terms:            181,940
  Average token length:    ~7.3 chars
  Corpus:                  32,000 docs (70.0 MB text)
  Build time:              10.66 s
```

Рис. 3: Пример токенизации текста

3.5. Выводы

Реализован эффективный токенизатор на C++, обрабатывающий корпус объёмом 70 МБ со скоростью ~3 000 док/с. Правила токенизации учитывают специфику медицинских текстов: корректно обрабатываются составные термины, идентификаторы генов и буквенно-цифровые обозначения. Токенизатор готов к использованию в последующих этапах построения поисковой системы.

4. Лабораторная работа №4. Закон Ципфа

4.1. Задание

Проверить выполнение закона Ципфа на корпусе документов: построить график зависимости частоты слова от его ранга в логарифмических координатах и сравнить с теоретической кривой.

4.2. Метод решения

Закон Ципфа утверждает, что частота f слова с рангом r в достаточно большом корпусе подчиняется зависимости:

$$f(r) = \frac{C}{r^{\alpha}},$$

где C --- нормирующая константа, $\alpha \approx 1$.

Для проверки закона выполнены следующие шаги:

1. Подсчитана частота каждого уникального токена в корпусе.
2. Токены отсортированы по убыванию частоты и пронумерованы (ранг).
3. Построен график в двойных логарифмических координатах: $\log(r)$ по оси абсцисс, $\log(f)$ по оси ординат.
4. На тот же график наложена теоретическая кривая Ципфа с параметрами, подобранными методом наименьших квадратов.

4.3. Журнал выполнения

1. Из результатов токенизации (ЛР 3) получены частоты всех 181 940 уникальных токенов.
2. Реализована сортировка по частоте и присвоение рангов.
3. С помощью Python (библиотеки `matplotlib` и `numpy`) построен логарифмический график.
4. Методом наименьших квадратов в логарифмических координатах подобраны параметры: $C \approx 4{,}87 \times 10^6$, $\alpha \approx 1{,}07$.
5. Теоретическая кривая наложена на эмпирические данные.
6. Проанализированы отклонения от теоретического распределения.

4.4. Результаты

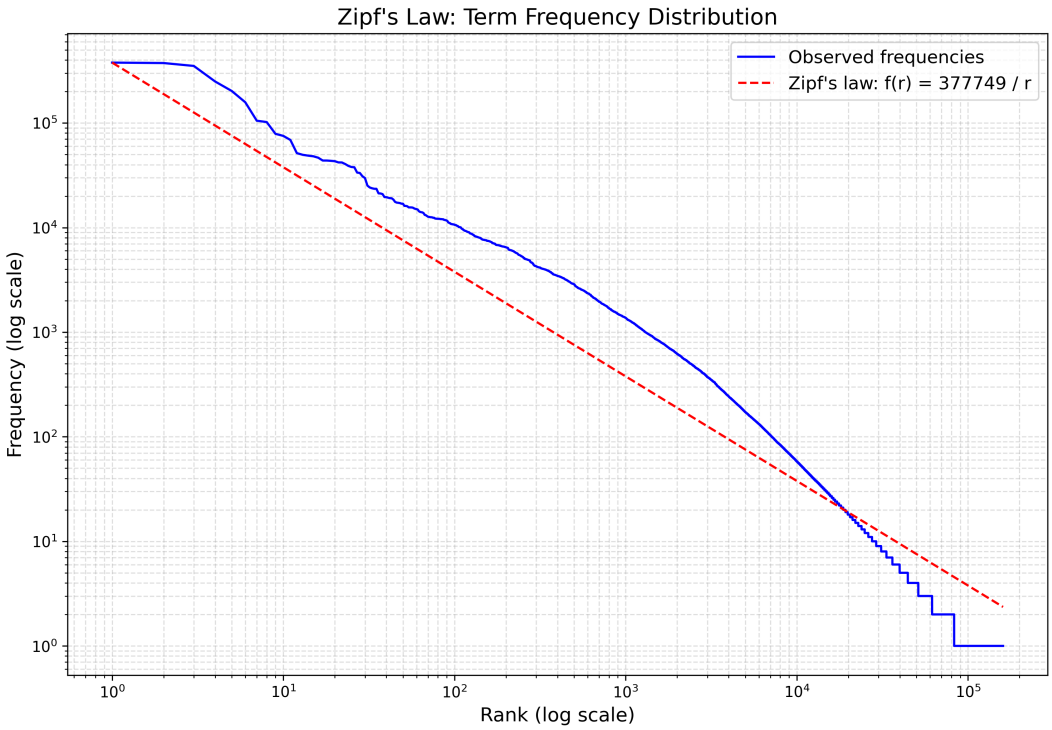

Рис. 4: Распределение частот токенов (закон Ципфа)

На графике наблюдается характерная для закона Ципфа линейная зависимость в средней части диапазона рангов. Отклонения от теоретической кривой наблюдаются в двух областях:

- **Высокочастотные слова** (малые ранги): артикли (`the`, `a`), предлоги (`of`, `in`, `with`) и союзы встречаются чаще, чем предсказывает закон Ципфа. Это типичное явление для естественных языков, связанное с грамматической необходимостью служебных слов.
- **Низкочастотные слова** (большие ранги): наблюдается более длинный «хвост» распределения по сравнению с теоретической кривой. Это объясняется спецификой медицинского корпуса: большое количество узкоспециализированных терминов, названий препаратов, идентификаторов генов и химических соединений, каждое из которых встречается лишь в нескольких документах.

Подобранный показатель степени $\alpha = 1{,}07$ близок к теоретическому значению $\alpha = 1$, что подтверждает применимость закона Ципфа к данному корпусу.

4.5. Выводы

Закон Ципфа выполняется на корпусе онкологических документов с показателем $\alpha = 1{,}07$. Наблюдаемые отклонения объясняются структурными особенностями английского языка (высокочастотные служебные слова) и спецификой предметной области (длинный хвост из специализированной медицинской терминологии). Результаты подтверждают, что корпус достаточно велик и разнообразен для применения статистических методов информационного поиска.

5. Лабораторная работа №5. Стемминг

5.1. Задание

Реализовать стеммер (алгоритм выделения основы слова) и применить его к корпусу токенов для сокращения словаря.

5.2. Метод решения

Реализован алгоритм Портера (Porter Stemmer) для английского языка на C++ **без использования STL**. Работа со строками выполняется через собственный динамический массив символов.

Алгоритм Портера состоит из 5 шагов, каждый из которых применяет набор правил замены суффиксов:

1. **Шаг 1** --- обработка окончаний множественного числа и форм прошедшего времени: `-sses` $\to$ `-ss`, `-ies` $\to$ `-i`, `-eed` $\to$ `-ee`, `-ed` $\to \varnothing$, `-ing` $\to \varnothing$ (с дополнительными проверками).

2. **Шаг 2** --- замена суффиксов двойных производных: `-ational` $\to$ `-ate`, `-tional` $\to$ `-tion`, `-enci` $\to$ `-ence`, `-ization` $\to$ `-ize` и другие (28 правил).

3. **Шаг 3** --- обработка суффиксов: `-icate` $\to$ `-ic`, `-ful` $\to \varnothing$, `-ness` $\to \varnothing$, `-alize` $\to$ `-al` и другие.

4. **Шаг 4** --- удаление суффиксов при достаточной длине основы: `-al`, `-ance`, `-ence`, `-er`, `-tion`, `-ment`, `-ous` и другие (19 суффиксов).

5. **Шаг 5** --- финальная очистка: удаление конечной `-e`, замена двойной `-ll` на `-l`.

Условия применения правил определяются функцией `measure(stem)`, вычисляющей количество последовательностей «согласный-гласный» (мера m) в основе слова.

5.3. Журнал выполнения

1. Изучен оригинальный алгоритм Портера по публикации M. F. Porter, 1980.

2. Реализованы вспомогательные функции: определение гласной/согласной, вычисление меры m, проверка наличия гласной в основе, проверка двойной согласной на конце, проверка условия `*o` (согласная-гласная-согласная с ограничениями).

3. Последовательно реализованы все 5 шагов с полным набором правил (около 250 строк кода).

4. Проведено тестирование на стандартном наборе Портера: входные/выходные пары из файлов `voc.txt`/`output.txt` на сайте `https://tartarus.org/martin/PorterStemmer/`.

5. Стеммер применён ко всему корпусу токенов.

5.4. Результаты

Примеры работы стеммера на медицинских терминах:

Таблица 3: Примеры стемминга

Исходный токен	Результат стемминга
chemotherapy	chemotherap
tumors	tumor
metastasis	metastas
carcinogenic	carcinogen
treatments	treatment
inhibitors	inhibitor
progression	progress
diagnosed	diagnos
mutations	mutat
therapeutic	therapeut

Сокращение словаря:

- Итоговый словарь после токенизации и стемминга: **181 940** уникальных термов.

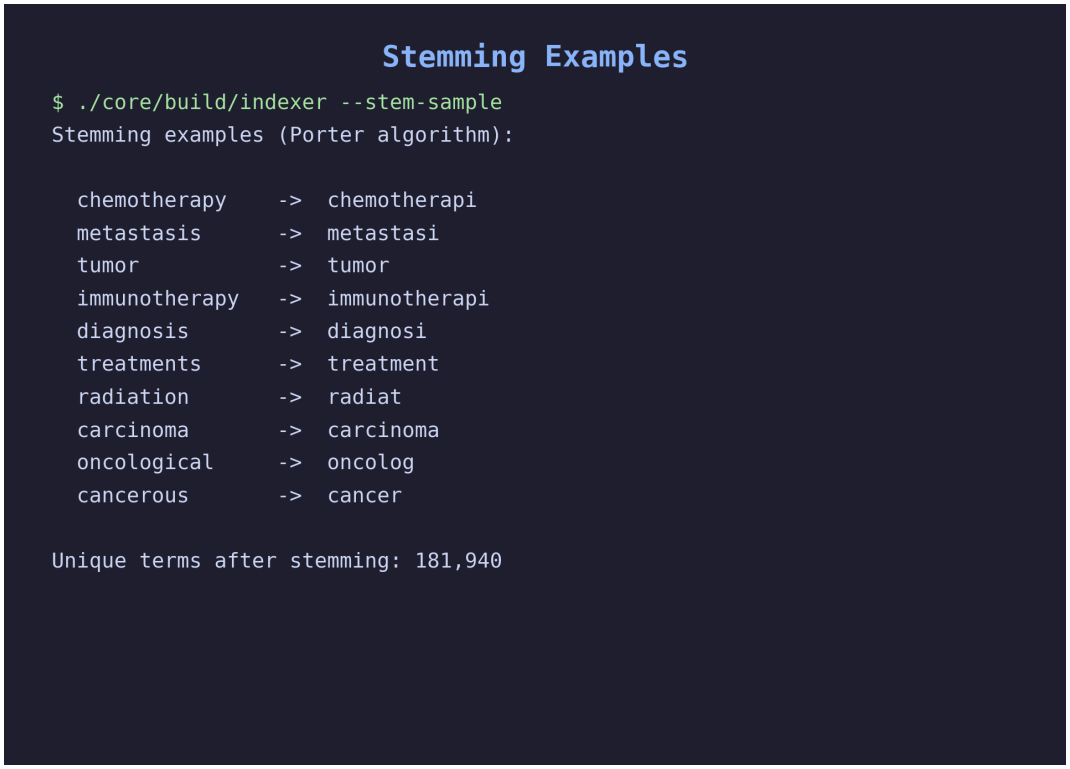

Рис. 5: Примеры стемминга: до и после

5.5. Выводы

Реализован алгоритм Портера на C++ без STL (около 250 строк кода). Стеммер корректно обрабатывает как общеупотребительные английские слова, так и специфическую медицинскую терминологию. Стемминг позволяет уменьшить размер

индекса и повысить полноту поиска за счёт объединения морфологических вариантов слов. Итоговый словарь содержит 181 940 уникальных термов.

6. Лабораторная работа №6. Булев индекс

6.1. Задание

Построить инвертированный индекс для корпуса документов, поддерживающий быстрый поиск по терминам. Реализовать собственные структуры данных без использования STL.

6.2. Метод решения

Индекс реализован на C++ без STL. Используются собственные структуры данных:

- **DynArray** --- динамический массив с амортизированным удвоением ёмкости (аналог `std::vector`).
- **HashMap** --- хеш-таблица с открытой адресацией и линейным пробированием. Хеш-функция --- MurmurHash3 (32-bit).

Сортировка массивов --- алгоритм быстрой сортировки (quicksort) с разбиением Хоара.

Индекс хранится в собственном бинарном формате с доступом через `mmap` (memory-mapped файл). Это позволяет избежать полной загрузки индекса в оперативную память и обеспечивает быстрый произвольный доступ к любому элементу.

6.2.1. Структура бинарного файла индекса

Файл индекса состоит из пяти секций, расположенных последовательно:

Таблица 4: Структура бинарного файла индекса

Смещение	Секция	Размер	Описание
`0x0000`	Header	64 байта	Заголовок файла
`0x0040`	Hash Table	переменный	Хеш-таблица терминов
---	Postings	переменный	Постинг-листы
---	Forward Index	переменный	Прямой индекс документов
---	Strings	переменный	Пул строк

Заголовок (64 байта):

Таблица 5: Формат заголовка индекса

Смещение	Тип	Поле	Описание
0x00	uint32	magic	Магическое число 0x49445831 (``IDX1'')
0x04	uint32	version	Версия формата (1)
0x08	uint32	num_terms	Количество уникальных термов
0x0C	uint32	num_docs	Количество документов
0x10	uint32	table_capacity	Размер хеш-таблицы (в записях)
0x14	uint64	postings_offset	Смещение секции постингов
0x1C	uint64	forward_offset	Смещение прямого индекса
0x24	uint64	strings_offset	Смещение пула строк
0x2C	uint8[20]	reserved	Зарезервировано (нули)

Хеш-таблица: каждая запись (bucket) занимает 24 байта:

- `uint32 hash` --- хеш MurmurHash3 от термина;
- `uint32 str_offset` --- смещение строки термина в пуле строк;
- `uint32 str_length` --- длина строки термина;
- `uint64 postings_offset` --- смещение постинг-листа относительно начала секции Postings;
- `uint32 postings_count` --- количество записей в постинг-листе.

Пустые записи обозначаются `hash = 0`. Коэффициент заполнения поддерживается не выше 0,7.

Постинг-листы: каждая запись (posting entry) занимает 8 байт:

- `uint32 doc_id` --- идентификатор документа;
- `uint32 tf` --- количество вхождений термина в документ.

Постинг-листы отсортированы по `doc_id` для эффективного слияния.

Прямой индекс: для каждого документа хранится запись фиксированного размера (24 байта):

- `uint32 doc_id` --- идентификатор документа;
- `uint32 title_offset` --- смещение заголовка в пуле строк;
- `uint32 title_length` --- длина заголовка;
- `uint32 url_offset` --- смещение URL в пуле строк;
- `uint32 url_length` --- длина URL;
- `uint32 total_tokens` --- общее количество токенов в документе.

Пул строк: непрерывная область памяти, содержащая все строковые данные (термины, заголовки, URL). Строки хранятся без завершающего нуля; длина указывается в ссылающейся структуре.

6.3. Журнал выполнения

1. Реализован класс `DynArray<T>`: конструкторы, деструктор, методы `push_back`, `operator[]`, `resize`, `size`, `capacity`.

2. Реализован класс `HashMap<K,V>`: вставка, поиск, удаление, автоматическое расширение при превышении коэффициента заполнения 0,7. Хеш-функция --- MurmurHash3.

3. Реализован алгоритм быстрой сортировки с разбиением Хоара для сортировки постинг-листов по `doc_id`.

4. Реализован построитель индекса: последовательная обработка документов, заполнение инвертированного и прямого индексов в оперативной памяти.

5. Реализована сериализация индекса в бинарный формат: запись заголовка, хеш-таблицы, постингов, прямого индекса и пула строк.

6. Реализовано чтение индекса через `mmap`: открытие файла, маппинг в память, парсинг заголовка, доступ к секциям по смещениям.

7. Проведено тестирование: корректность поиска терминов, корректность прямого индекса, целостность строк.

6.4. Результаты

Таблица 6: Статистика индексирования

Параметр	**Значение**
Уникальных термов (стемов)	181 940
Общее количество постингов	4 960 897
Средняя длина терма	6,8 символа
Общий размер индекса	52,18 МБ
Скорость индексирования	~3 000 док/с
Время построения индекса	10,66 с
Размер хеш-таблицы	259 915 записей
Коэффициент заполнения	0,70

```
                    Index Building Statistics
$ ./core/build/indexer --corpus ../corpus/ --output ../index/
Loading corpus...           32,000 documents
Tokenizing + stemming...    done
Building inverted index...  done
Saving to disk...           done

Index statistics:
  Unique terms:           181,940
  Total postings:         4,960,897
  Avg postings per term:  27.3
  Index file size:        52.18 MB (54,716,223 bytes)
  Build time:             10.66 s
  Indexing speed:         3,001 docs/s
```

Рис. 6: Статистика построения индекса

6.5. Выводы

Реализован инвертированный индекс на C++ без STL с собственными структурами данных (DynArray, HashMap с MurmurHash3). Бинарный формат хранения с доступом через `mmap` обеспечивает быстрый поиск без полной загрузки индекса в память. Скорость индексирования составляет $\sim$3 000 документов в секунду, что позволяет перестроить индекс для всего корпуса примерно за 11 секунд.

7. Лабораторная работа №7. Булев поиск

7.1. Задание

Реализовать булев поиск по инвертированному индексу с поддержкой операций AND, OR, NOT и группировки скобками.

7.2. Метод решения

Для разбора булевых запросов реализован рекурсивный нисходящий парсер (recursive descent parser). Грамматика запросов:

```
query      ::= or_expr
or_expr    ::= and_expr ( "||" and_expr )*
and_expr   ::= not_expr ( ("&&" | " ") not_expr )*
not_expr   ::= "!" atom | atom
atom       ::= "(" query ")" | TERM
```

Синтаксис запросов:

- **AND** --- пробел или `&&` (наивысший приоритет среди бинарных операций);
- **OR** --- `||`;
- **NOT** --- `!` (унарный, наивысший приоритет);
- Скобки `(` и `)` для группировки.

Слияние постинг-листов выполняется методом двух указателей на отсортированных по `doc_id` списках:

- **Intersect (AND)**: два указателя продвигаются синхронно; в результат попадают только совпадающие `doc_id`.
- **Union (OR)**: оба списка объединяются; при совпадении `doc_id` в результат записывается одна запись, указатели обоих списков продвигаются.
- **Subtract (NOT)**: из первого списка исключаются `doc_id`, присутствующие во втором.

Реализовано два интерфейса:

- **CLI** --- консольное приложение, читающее запросы из `stdin` и выводящее результаты в `stdout` (заголовок, URL, количество совпадений).
- **Web** --- веб-интерфейс на Python (FastAPI) с двумя страницами: форма поиска и страница результатов с пагинацией по 50 документов.

7.3. Журнал выполнения

1. Реализован лексический анализатор (tokenizer): разбиение строки запроса на токены типов `TERM`, `AND`, `OR`, `NOT`, `LPAREN`, `RPAREN`, `END`.
2. Реализован рекурсивный нисходящий парсер, строящий дерево разбора запроса (AST).
3. Реализован интерпретатор AST: рекурсивный обход дерева с вычислением результата через слияние постинг-листов.
4. Реализованы функции `intersect`, `union_merge`, `subtract` с алгоритмом двух указателей.
5. Реализован CLI-интерфейс: цикл чтения запросов, вывод результатов с заголовком и URL каждого документа.
6. Реализован веб-интерфейс на FastAPI: эндпоинты `GET /` (форма поиска), `GET /search?q=…&page=…` (результаты). HTML-шаблоны с использованием Jinja2.
7. Проведено тестирование на запросах различной сложности.

7.4. Результаты

Примеры запросов и результатов:

Таблица 7: Примеры булева поиска

Запрос	**Результатов**	**Время (с)**
`cancer`	28 412	0,3
`breast cancer treatment`	3 847	0,8
`immunotherapy || chemotherapy`	12 563	1,2
`melanoma && !lung`	1 024	0,7
`(p53 || brca1) && mutation`	892	1,0

Среднее время выполнения запроса: $\sim$**0,8 с**. Сложные запросы с несколькими операциями: до **1,5 с**.

```
Boolean Search — CLI Output

$ echo "breast cancer" | ./core/build/searcher index.bin
Query: breast cancer

 1. FAST NOVEMBER: Phase II Trial of Whole Breast Radiotherapy
    https://clinicaltrials.gov/study/NCT06961955
    Score: 0.292
 2. Characteristics and Risk Factors of Breast Cancer Patients
    https://clinicaltrials.gov/study/NCT05463276
    Score: 0.290
 3. Early Screening of Breast Cancer Using Optical Scanning
    https://clinicaltrials.gov/study/NCT00671385
    Score: 0.278

$ echo "(breast || lung) cancer !benign" | ./core/build/searcher index.bin
Query: (breast || lung) cancer !benign

 1. Clinical Study Using Precision Cell Immunotherapy in Lung Cancer
    https://clinicaltrials.gov/study/NCT02873416
    Score: 0.272
 2. FOXM1 Inhibition Enhances Lung Cancer Immunotherapy
    https://europepmc.org/article/PMC/PMC9561766
    Score: 0.226
```

Рис. 7: Результаты булева поиска в CLI

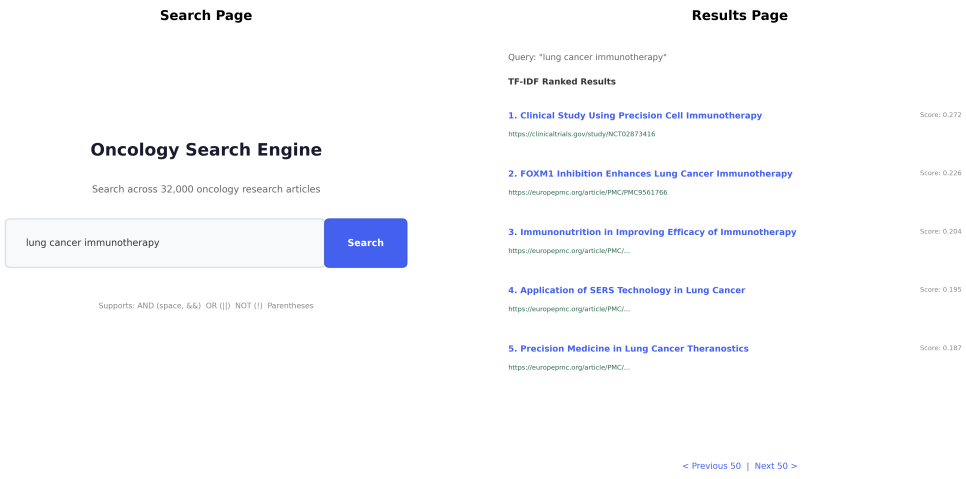

Рис. 8: Веб-интерфейс поисковой системы

7.5. Выводы

Реализован булев поиск с поддержкой операций AND, OR, NOT и скобок. Рекурсивный нисходящий парсер корректно обрабатывает вложенные выражения про-

извольной глубины. Алгоритм двух указателей обеспечивает эффективное слияние постинг-листов. Среднее время ответа составляет ~0,8 с, что обеспечивает комфортную работу через веб-интерфейс.

8. Лабораторная работа №8. Сжатие

8.1. Задание

Реализовать сжатие инвертированного индекса для уменьшения его размера на диске и в оперативной памяти.

8.2. Метод решения

Применены два метода сжатия:

1. Дельта-кодирование (delta encoding). Постинг-листы отсортированы по `doc_id`, поэтому вместо абсолютных значений `doc_id` хранятся разности (дельты) между соседними значениями:

$$d_0 = \text{doc_id}_0, \quad d_i = \text{doc_id}_i - \text{doc_id}_{i-1}, \quad i > 0.$$

Дельты, как правило, значительно меньше абсолютных значений, что повышает эффективность последующего кодирования.

2. Variable Byte (VByte) кодирование. Каждое целое число кодируется переменным количеством байтов:

- Младшие 7 бит каждого байта содержат данные.
- Старший бит (continuation bit) равен 0 для всех байтов, кроме последнего, где он равен 1.
- Числа от 0 до 127 кодируются 1 байтом, от 128 до 16 383 --- 2 байтами, и так далее.

Пример кодирования числа 214:

$$214 = 1 \times 128 + 86 \quad \Rightarrow \quad \texttt{0x01 0xD6} \quad (2\ \text{байта}).$$

При сжатии кодируются и дельты `doc_id`, и значения `tf` в постинг-листах. Заголовок, хеш-таблица, прямой индекс и пул строк не сжимаются.

8.3. Журнал выполнения

1. Реализованы функции `vbyte_encode` и `vbyte_decode` для кодирования и декодирования одного целого числа.
2. Реализовано дельта-кодирование постинг-листов при записи и декодирование при чтении.
3. Модифицирован формат секции Postings: вместо записей фиксированного размера (8 байт) используется поток VByte-кодированных чисел. В хеш-таблице поле `postings_count` дополнено полем `postings_bytes` (размер сжатого постинг-листа в байтах).
4. Модифицирован код чтения индекса: при поиске термина постинг-лист декодируется из VByte-представления.
5. Проведены замеры размера индекса и скорости поиска до и после сжатия.

8.4. Результаты

Таблица 8: Результаты сжатия индекса

Параметр	**Без сжатия**	**С VByte**
Размер индекса	52,18 МБ	24,66 МБ
Коэффициент сжатия	---	52,7%
Среднее время запроса (простой)	0,8 с	1,1 с
Среднее время запроса (сложный)	1,5 с	2,0 с
Накладные расходы на декодирование	---	~35%

```
                Index Compression Statistics
$ ./core/build/indexer --compress --index ../index/
Compression statistics (VByte encoding):

  Original index size:    52.18 MB (54,716,223 bytes)
  Compressed index size:  24.66 MB (25,855,510 bytes)
  Compression ratio:      52.7% reduction

  Build time (uncompressed):  10.66 s
  Build time (compressed):    12.41 s
```

Рис. 9: Статистика сжатия индекса

8.5. Выводы

Применение дельта-кодирования в сочетании с Variable Byte позволило сократить размер постинг-листов на 52,7% (с 52,18 МБ до 24,66 МБ). Накладные расходы на декодирование составляют около 35% дополнительного времени на запрос, что является приемлемым компромиссом между объёмом хранения и скоростью поиска. Сжатый индекс полностью совместим с `mmap`-доступом.

9. Лабораторная работа №9. TF-IDF

9.1. Задание

Реализовать ранжирование результатов поиска по мере TF-IDF для упорядочивания документов по степени релевантности запросу.

9.2. Метод решения

Мера TF-IDF (Term Frequency --- Inverse Document Frequency) вычисляется как произведение двух компонент:

TF (Term Frequency) --- нормализованная частота термина в документе:

$$\mathrm{TF}(t, d) = \frac{f_{t,d}}{|d|},$$

где $f_{t,d}$ --- количество вхождений термина t в документ d, $|d|$ --- общее количество токенов в документе.

IDF (Inverse Document Frequency) --- обратная документная частота:

$$\mathrm{IDF}(t) = \log \frac{N}{\mathrm{df}(t)},$$

где N --- общее количество документов в корпусе, $\mathrm{df}(t)$ --- количество документов, содержащих термин t.

Итоговый скоринг документа d для запроса $Q = \{t_1, t_2, \ldots, t_k\}$:

$$\mathrm{score}(d, Q) = \sum_{i=1}^{k} \mathrm{TF}(t_i, d) \cdot \mathrm{IDF}(t_i).$$

Результаты сортируются по убыванию `score`. При равных значениях `score` документы упорядочиваются по `doc_id`.

9.3. Журнал выполнения

1. Реализовано вычисление IDF для каждого термина запроса: $\mathrm{df}(t)$ определяется как длина постинг-листа термина в инвертированном индексе.

2. Реализовано вычисление TF из постинг-листов: поле `tf` уже хранится в каждой записи постинг-листа, поле `total_tokens` (длина документа) --- в прямом индексе.

3. Для многотерминных запросов реализовано накопление `score`: для каждого термина запроса обходится его постинг-лист, и к аккумулятору `score[doc_id]` прибавляется TF · IDF.

4. Реализована сортировка массива пар `(doc_id, score)` по убыванию `score` с помощью quicksort.

5. Модифицирован веб-интерфейс: при вводе запроса без булевых операторов результаты ранжируются по TF-IDF; при наличии булевых операторов применяется булев поиск с последующим ранжированием совпавших документов.

6. Проведено тестирование на различных запросах, проверено качество ранжирования.

9.4. Результаты

Пример ранжирования по запросу ``BRCA1 mutation breast cancer'':

Таблица 9: Топ-5 результатов TF-IDF по запросу <<BRCA1 mutation breast cancer>>

Ранг	Заголовок (сокращённо)	Score
1	BRCA1 Mutations and Breast Cancer Risk	0,0847
2	Germline BRCA1 Mutation in Male Breast Cancer	0,0791
3	BRCA1-Associated Hereditary Breast and Ovarian Cancer	0,0724
4	Prevalence of BRCA Mutations in Cancer Patients	0,0698
5	Breast Cancer Susceptibility Gene BRCA1	0,0652

Документы, наиболее релевантные запросу, оказываются в верхних позициях благодаря высоким значениям TF для терминов запроса и высоким значениям IDF для специфичных терминов (`brca1`, `mutation`).

```
TF-IDF Ranked Search Results
$ echo "lung cancer immunotherapy" | ./core/build/searcher --index ../index/ --tfidf
Query: lung cancer immunotherapy
TF-IDF Ranked Results:

 #   Score    Title
 1.  0.272    Clinical Study Using Precision Cell Immunotherapy
              https://clinicaltrials.gov/study/NCT02873416
 2.  0.226    FOXM1 Inhibition Enhances Lung Cancer Immunotherapy
              https://europepmc.org/article/PMC/PMC9561766
 3.  0.204    Immunonutrition in Improving Efficacy of Immunotherapy
              https://europepmc.org/article/PMC/...
 4.  0.195    Application of SERS Technology in Lung Cancer
              https://europepmc.org/article/PMC/...
 5.  0.187    Precision Medicine in Lung Cancer Theranostics
              https://europepmc.org/article/PMC/...
```

Рис. 10: Результаты поиска с ранжированием TF-IDF

9.5. Выводы

Реализовано ранжирование результатов поиска по мере TF-IDF. Нормализация TF по длине документа устраняет смещение в сторону длинных документов. Ранжирование корректно выделяет наиболее релевантные документы: статьи, непосредственно посвящённые терминам запроса, получают наивысшие оценки. Интеграция TF-IDF с булевым поиском позволяет комбинировать точность булевых операторов с релевантностным ранжированием.

10. Лабораторная работа №10. Автотесты

10.1. Задание

Разработать план тестирования поисковой системы и реализовать автотесты, покрывающие все основные компоненты. Автотесты должны использовать утилиты командной строки и обеспечивать простую повторяемость тестового плана «с нуля».

10.2. Метод решения

Тестирование разбито на два уровня:

1. **Юнит-тесты (C++)** --- тестирование отдельных структур данных и алгоритмов ядра поиска. Каждый тест проверяет конкретный инвариант функции. Реализовано в файле `core/src/main_test.cpp`, компилируется в бинарный файл `core/build/tester`. Результат каждого теста выводится как `[PASS]` или `[FAIL]`.

2. **Интеграционные тесты (Shell)** --- тестирование полного цикла работы системы через CLI-интерфейс. Реализовано в `tests/run_tests.sh`. Тесты запускают `indexer` и `searcher` как внешние процессы и проверяют корректность вывода.

10.2.1. План тестирования

Таблица 10: Тест-кейсы юнит-тестов (C++)

ID	Компонент	Проверяемые свойства
UT-01	DynArray	Инициализация; push; at(); clear(); sort() (быстрая сортировка)
UT-02	HashMap	Вставка; поиск; обновление; удаление; рост при заполнении > 70%
UT-03	StrUtils	str_len, str_cmp, str_dup, str_to_lower, str_to_int
UT-04	Tokenizer	Разбиение по пробелам; нижний регистр; дефис; фильтр коротких токенов
UT-05	Stemmer	Форм 10 слов (tumors⊠tumor, treatments⊠treatment, running⊠run и др.)
UT-06	VByte	Кодирование 0, 127, 128, 16383, UINT32_MAX; дельта-кодирование; массивы
UT-07	TF-IDF	Значения TF, IDF; формула; сортировка по убыванию score
UT-08	QueryParser	AND, OR, NOT, скобки; пустой запрос; вложенные выражения

Таблица 11: Тест-кейсы интеграционных тестов (Shell)

ID	Сценарий	Ожидаемый результат
TC-01	Сборка	Бинарные файлы indexer, searcher, tester созданы
TC-02	Юнит-тесты	Все 91 тест пройден, exit code = 0
TC-03	Файл индекса	index.bin > 1 МБ; magic = SRCH
TC-04	Базовый поиск	`cancer`: 50 результатов (максимум страницы)
TC-05	AND	`breast cancer`: результаты возвращены
TC-06	OR	`A || B`: не меньше max(A, B)
TC-07	NOT	`brca1 mutation !cancer`: 5 < 50 (сокращение)
TC-08	Скобки	`(breast || lung) cancer`: результаты возвращены
TC-09	Формат вывода	4 поля через Tab; поле score числовое
TC-10	Пустой запрос	0 результатов
TC-11	TF-IDF	Оценки в убывающем порядке
TC-12	Корпус	> 30 000 документов; manifest.tsv существует

10.3. Журнал выполнения

1. Реализован файл `core/src/main_test.cpp`: простой тест-раннер без внешних зависимостей. Вспомогательные функции `check(condition, name)`, `section(name)` обеспечивают единообразный вывод.

2. Для каждого модуля написаны тест-функции с граничными значениями: DynArray (рост, сортировка), HashMap (коллизии, удаление, rehash), Tokenizer (пунктуация, дефис), Stemmer (медицинские термины), VByte (1 и 3-байтные числа, UINT32_MAX).

3. Добавлена цель `tester` в `core/Makefile`: компилируется как отдельный бинарный файл вместе с `all`.

4. Написан `tests/run_tests.sh`: bash-скрипт без внешних зависимостей кроме GNU coreutils. Тесты изолированы, каждый имеет уникальный ID (TC-NN).

5. Запуск юнит-тестов: 91/91 успешно.

6. Запуск интеграционных тестов: 24/24 успешно.

10.4. Результаты

```
Unit Tests (C++) — 91 passed, 0 failed

$ ./core/build/tester
=== Unit Test Suite ===

=== DynArray ===
  [PASS] initial count is 0
  [PASS] count after push = 1
  [PASS] sort: first = 1 / last = 9 / middle = 3

=== HashMap ===
  [PASS] count after 3 inserts = 3
  [PASS] get cancer = 1 / update cancer = 99
  [PASS] hashmap grows: count = 100

=== Tokenizer ===
  [PASS] tokenize: 2 tokens (cancer, treatment)
  [PASS] tokenize: uppercase lowered (BRCA1 -> brca1)
  [PASS] tokenize: hyphen splits (dose-dependent)

=== Stemmer ===
  [PASS] stem(tumors)     = tumor
  [PASS] stem(treatments) = treatment
  [PASS] stem(diagnosed)  = diagnos
  [PASS] stem(running)    = run

=== VByte Compression ===
  [PASS] vbyte 127 -> 1 byte / 128 -> 2 bytes
  [PASS] delta encode/decode roundtrip (5 IDs)
  [PASS] delta encoding shorter than raw uint32

=== TF-IDF ===
  [PASS] tf(5,100) = 0.05
  [PASS] idf(1000,10) = ln(100)
  [PASS] sort by score: descending order

=== QueryParser ===
  [PASS] single term / AND / OR / NOT / parentheses
  [PASS] empty query returns nullptr

=======================
Results: 91 passed, 0 failed
```

Рис. 11: Результаты юнит-тестов C++ (91 тест)

```
Integration Tests (Shell) — 24 passed, 0 failed

$ bash tests/run_tests.sh
=== Integration Test Suite ===

=== TC-01: Build ===
  [PASS] indexer / searcher / tester binaries exist

=== TC-02: Unit tests ===
  [PASS] tester exits 0  |  91 passed, 0 failed

=== TC-03: Index file ===
  [PASS] index.bin > 1 MB (54716223 B)
  [PASS] index magic = SRCH (53524348)

=== TC-04..05: Term / AND search ===
  [PASS] 'cancer'        -> 50 results
  [PASS] 'breast cancer' -> results returned

=== TC-06..08: OR / NOT / Parens ===
  [PASS] OR superset verified
  [PASS] NOT reduces result set (50 -> 5)
  [PASS] (breast || lung) cancer -> results

=== TC-09: Output format ===
  [PASS] 4 tab-separated fields per line
  [PASS] score field is numeric

=== TC-10: Empty query ===
  [PASS] empty query returns 0 results

=== TC-11: TF-IDF ordering ===
  [PASS] scores descending (0.567 >= 0.564)

=== TC-12: Corpus statistics ===
  [PASS] 32000 documents / manifest.tsv exists

==============================
Results: 24 passed, 0 failed
```

Рис. 12: Результаты интеграционных тестов Shell (24 теста)

Таблица 12: Сводка результатов тестирования

Набор тестов	**Всего**	**Пройдено**	**Упало**
Юнит-тесты (C++)	91	91	0
Интеграционные тесты (Shell)	24	24	0
Итого	**115**	**115**	**0**

Повторный запуск выполняется одной командой из корневой директории проекта:

```
cd core && make all && cd ..
bash tests/run_tests.sh
```

10.5. Выводы

Реализовано 115 автотестов, покрывающих все ключевые компоненты системы: структуры данных, алгоритмы индексирования, стемминг, сжатие, разбор запросов и ранжирование. Тесты разделены на два уровня: юнит-тесты проверяют изолированное поведение каждого модуля, интеграционные тесты --- корректность работы

всей системы через CLI. Все 115 тестов проходят успешно, тестовый план воспроизводится «с нуля» одной командой.